

CHAPTER – 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

INTRODUCTION

The Meesho Women's Fashion E-Commerce Database Management System is designed to efficiently manage customers, sellers, product categories, products, orders, payments, and delivery information. Functional dependencies help identify the relationship between attributes in each table, while redundancy analysis and data anomaly detection ensure data consistency and reduce duplication. A well-designed database improves performance, maintains data integrity, and supports efficient management of the Meesho women's fashion e-commerce platform.

1. CUSTOMER TABLE

Functional Dependency

Customer_ID → Customer_Name, Age, Email, Phone_Number, Address

Redundancy Analysis

Customer information is stored only once in the Customer table. Orders store only the Customer_ID, preventing duplicate customer records.

Data Anomalies

- **Insert Anomaly:** A new customer can be added without placing an order.
- **Update Anomaly:** Updating a customer's email, phone number, or address requires changes only in the Customer table.
- **Delete Anomaly:** Deleting an order will not remove customer information.

2. SELLER TABLE

Functional Dependency

Seller_ID → Seller_Name, Email, Phone_Number

Redundancy Analysis

Seller information is stored only once in the Seller table. Products store only the Seller_ID, avoiding duplicate seller records.

Data Anomalies

- **Insert Anomaly:** A new seller can be added without adding products.
- **Update Anomaly:** Updating seller details requires changes only in the Seller table.
- **Delete Anomaly:** Deleting a product will not remove seller information.

3. CATEGORY TABLE

Functional Dependency

Category_ID → Category_Name

Redundancy Analysis

Each category is stored only once. Products reference the category using Category_ID instead of repeating the category name.

Data Anomalies

- **Insert Anomaly:** New categories can be added without products.
- **Update Anomaly:** Renaming a category is done only once.
- **Delete Anomaly:** Deleting a product does not delete the category.

4. PRODUCT TABLE

Functional Dependency

Product_ID → Product_Name, Price, Stock, Seller_ID, Category_ID

Redundancy Analysis

Product details are stored only in the Product table. Orders reference products using Product_ID, preventing repeated product information.

Data Anomalies

- **Insert Anomaly:** New products can be added before receiving orders.
- **Update Anomaly:** Product price or stock is updated only once.
- **Delete Anomaly:** Deleting an order does not remove product information.

5. ORDERS TABLE

Functional Dependency

Order_ID → Customer_ID, Product_ID, Order_Date, Total_Amount, Order_Status

Redundancy Analysis

The Orders table stores only order-related information. Customer and product details are referenced using foreign keys.

Data Anomalies

- **Insert Anomaly:** Orders can be created independently using existing Customer_ID and Product_ID.
- **Update Anomaly:** Order status can be updated without affecting other tables.
- **Delete Anomaly:** Deleting an order does not remove customer or product data.

6. PAYMENT TABLE

Functional Dependency

Payment_ID → Order_ID, Payment_Mode, Payment_Status

Redundancy Analysis

Payment information is stored separately from the Orders table to avoid duplicate payment records.

Data Anomalies

- **Insert Anomaly:** Payment details can be recorded after creating an order.
- **Update Anomaly:** Payment status is updated only once.
- **Delete Anomaly:** Deleting payment information does not remove the order.

7. DELIVERY TABLE

Functional Dependency

Delivery_ID → Order_ID, Delivery_Partner, Delivery_Status

Redundancy Analysis

Delivery details are maintained in a separate table, preventing repeated delivery information.

Data Anomalies

- **Insert Anomaly:** Delivery records can be added after order confirmation.
- **Update Anomaly:** Delivery status can be updated independently.
- **Delete Anomaly:** Deleting delivery information does not affect order details.

CONCLUSION

The Meesho Women's Fashion E-Commerce Database Management System is designed using proper functional dependencies and normalized tables. Each entity stores only its relevant data, reducing redundancy and preventing insert, update, and delete anomalies. This design improves data consistency, maintains integrity, and ensures efficient database management for the Meesho women's fashion e-commerce platform.